Guion PHP2Shell · IS-1023

Guion de exposición · PHP2Shell (IS-1023)

Tono: profesional, claro y directo. **Narración:** audio neural en español, 15–16 minutos (31 diapositivas).

> Regla: la narración no lee comandos símbolo por símbolo; explica qué hace cada paso. Los comandos quedan visibles en la diapositiva y se ejecutan en la réplica.

Mapa rápido

#	Diapositiva	Expone
01	Portada — PHP2Shell	Equipo completo
02	Cinco fases, una historia	Equipo completo
03	Contexto: qué quisimos montar	M1 · Erick Ramos Vera
04	Arquitectura y camino de ataque	M1 · Erick Ramos Vera
05	Qué tiene cada máquina	M1 · Erick Ramos Vera
06	M1 · Aplicación	M1 · Erick Ramos Vera
07	M2 · Base de datos	M2 · Doris Apaza Anahua
08	M3 · Respaldos	M3 · Rony Churata Huacani
09	M4 · Windows Server	M4 · Fabiola Ancco Rafael
10	M5 · Cliente	M5 · Marco Nina Cruz
11	M6 · Auditoría	M6 · Jose Paye Mamani
12	Reparto de PHP2Shell	Equipo completo
13	Metodología y version gate	M1 · Erick Ramos Vera
14	Qué es PHP2Shell y por qué importa	M1 · Erick Ramos Vera
15	W2 · Ficha	M1 · Erick Ramos Vera
16	W2 · Version gate	M1 · Erick Ramos Vera
17	W2 · Precondiciones	M2 · Doris Apaza Anahua
18	W2 · Rebuild	M2 · Doris Apaza Anahua
19	W2 · Configuración y offsets	M3 · Rony Churata Huacani
20	W2 · Chain A	M4 · Fabiola Ancco Rafael
21	W2 · Comandos	M5 · Marco Nina Cruz
22	Otra vía · W3	M1 · Erick Ramos Vera

#	Diapositiva	Expone
23	Otra vía · B3	M3 · Rony Churata Huacani
24	W2 · Evidencia	M6 · Jose Paye Mamani
25	W2 · Impacto	M6 · Jose Paye Mamani
26	Cadena completa: de Kali a root	M1 Erick · M3 Rony · M6 Jose
27	Resto de la red	M2 Doris · M3 Rony · M4 Fabiola · M5 Marco
28	Remediación	Equipo completo
29	Retest: antes y después	Equipo completo
30	Cuatro lecciones de PHP2Shell	Equipo completo
31	Cierre y preguntas	Equipo completo

01 · Portada — PHP2Shell

Expone: Equipo completo
Voz: Equipo
Duración: 24.8 s
Idea clave: PHP2Shell: de PHP 8.4.26 a root, con evidencia.

Narración: Buenas tardes. Somos el equipo de Seguridad de la Información y presentamos PHP2Shell, un caso real de explotación sobre PHP 8.4.26 en la red universitaria. Partimos de la superficie expuesta, llegamos a www-data y luego a root. Todo está respaldado con comandos, capturas y hashes. La réplica se hará en vivo desde Kali.

Apoyo visual: Saludar, presentar al equipo y señalar el título, los dos CVE y el target session2.php en el puerto 8080.

Transición: Pasamos al recorrido de la exposición.

---

02 · Cinco fases, una historia

Expone: Equipo completo
Voz: Equipo
Duración: 36.5 s
Idea clave: Cinco fases: configuración, exploración, W2, mitigación y retest.

Narración: El recorrido tiene cinco fases. Fase uno: configuración y superficie, con la red, topología, activos, puertos y versiones. Fase dos: exploración, con dos comandos por máquina y su salida. Fase tres: explotación de PHP2Shell, desde el version gate hasta Chain A, www-data y root. Fase cuatro: mitigación, con versión o configuración por máquina y el comando de verificación. Fase cinco: retest y cierre. La idea es simple: de la versión vulnerable a root, con evidencia en cada fase.

Apoyo visual: Recorrer las cinco fases; detenerse en la fase 3, que es el caso PHP2Shell.

Transición: Empezamos por la fase 1.

03 · Contexto: qué quisimos montar

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 29.3 s
Idea clave: Red universitaria, servicios reales y un atacante dentro.

Narración: La red universitaria expone un portal web, base de datos, respaldos, un Windows heredado y estaciones de trabajo. Todo en 10.10.10.0/24; M6 es el punto de auditoría desde donde se valida la superficie. Identificamos veinticuatro hallazgos; veintidós se llevaron a evidencia. El objetivo es identificar, enumerar, explotar, escalar a root, remediar y verificar.

Apoyo visual: Señalar la red, los seis servidores y los tres números: 24 hallazgos, 22 con evidencia y 6 servidores.

Transición: Veamos cómo se conectan esas máquinas.

---

04 · Arquitectura y camino de ataque

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 32.5 s
Idea clave: Una sola puerta web; pivote desde M1.

Narración: La topología tiene una sola puerta web: M1. Desde M1 se pivota a M2, la base de datos, y a M3, los respaldos. M4 es el Windows Server y M5 la estación cliente. R0 solo transporta. La separación es lógica, no física: documentamos que 445 de M4 y 5900 de M5 quedaron visibles desde M6. Ese detalle también es parte del hallazgo.

Apoyo visual: Seguir el camino M6 → M1:80 → M2/M3/M4 y M5 por VNC; señalar la nota de firewall.

Transición: Estas son las versiones que encontramos.

---

05 · Qué tiene cada máquina

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 38.1 s
Idea clave: Versiones y puertos que activan el version gate.

Narración: Estas son las versiones que observamos desde Kali. M1 corre Apache 2.4.49, PHP 8.4.26 y WordPress 7.0.1. M2 tiene MariaDB 10.11.19 y Redis 6.2.6. M3 expone FTP, SMB y NFS. M4 es Windows Server 2016 con DNS y SMB. M5 es Windows 10 con TightVNC. Las versiones exactas son las que disparan el version gate.

Apoyo visual: Leer solo las versiones clave; no detenerse en cada puerto. M1 es la prioridad.

Transición: Ahora revisamos cada máquina y qué se explotó en ella.

---

06 · M1 · Aplicación

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 27.3 s
Idea clave: M1: W2 Chain A, W3, W4 y root por SSH/sudo.

Narración: M1 es el servidor de aplicación y la puerta de entrada. Publica WordPress y Moodle, y concentra la base de datos. Aquí explotamos W2 por Chain A, que nos dio www-data; W3, el traversal de Apache, que también dio www-data; y W4, una clave débil que permitió entrar como administrador. Con SSH y sudo cerramos el camino a root.

Apoyo visual: Señalar la tabla de vulns: W2, W3, W4 y M1-SSH; aclarar que W2 y W3 llegan a www-data y root se cierra por SSH/sudo.

Transición: La base de datos es el siguiente eslabón.

---

07 · M2 · Base de datos

Expone: M2 · Doris Apaza Anahua
Voz: Doris
Duración: 20.4 s
Idea clave: M2: Redis sin auth y volcado de bases.

Narración: M2 es la base de datos. Redis respondió sin autenticación y confirmamos el acceso con un PONG. Desde wp-config obtuvimos las claves de la base y, por un túnel SSH, leímos wordpress con doce tablas y moodle_db con cuatrocientas ochenta y siete. No hizo falta un exploit complejo: la configuración expuesta fue suficiente.

Apoyo visual: Señalar D1, D2 y D3; explicar que el dato sensible es el volcado, no solo el puerto abierto.

Transición: Los respaldos guardaban la llave que necesitábamos.

---

08 · M3 · Respaldos

Expone: M3 · Rony Churata Huacani
Voz: Rony
Duración: 23.2 s
Idea clave: M3: FTP anónimo y NFS entregan la llave.

Narración: M3 es el servidor de respaldos. El FTP aceptaba acceso anónimo y dentro estaba credenciales.txt con la clave Admin13. Esa misma clave se reutilizaba en M1. Además, NFS permitía escribir como root por no_root_squash. Con esas dos debilidades, M3 se convirtió en el eslabón que entregó la llave maestra.

Apoyo visual: Señalar B3 y B4; insistir en que no es un exploit nuevo, es reutilización de credenciales.

Transición: Pasamos a Windows Server.

---

09 · M4 · Windows Server

Expone: M4 · Fabiola Ancco Rafael
Voz: Fabiola
Duración: 24.4 s
Idea clave: M4: admin débil, AXFR y SYSTEM.

Narración: M4 es el Windows Server. La cuenta de administrador tenía una clave débil; con ella obtuvimos Pwn3d y acceso a las comparticiones. El DNS permitía transferencia de zona, así que obtuvimos el mapa completo de uni.lab. Con credenciales administrativas llegamos a SYSTEM mediante una tarea programada. No fue un privilege escalation desde un usuario limitado.

Apoyo visual: Señalar S3 y S4; aclarar que SYSTEM se alcanzó con credenciales administrativas, no desde un usuario común.

Transición: La estación cliente es el impacto sobre el usuario.

---

10 · M5 · Cliente

Expone: M5 · Marco Nina Cruz
Voz: Marco
Duración: 23.0 s
Idea clave: M5: VNC sin auth y escritorio comprometido.

Narración: M5 es la estación cliente. TightVNC no pedía autenticación, así que abrimos el escritorio directamente. whoami confirmó el usuario cliente y dejamos el marcador PWNED-M5. También se reutilizaba la clave por SMB. Es el impacto más visible: el usuario final, su escritorio y sus archivos. La remediación de M5 quedó pendiente por filtrado.

Apoyo visual: Señalar C4 y C2; mostrar que M5 está marcado como pendiente, no como remediado.

Transición: M6 es el punto de partida y la evidencia.

---

11 · M6 · Auditoría

Expone: M6 · Jose Paye Mamani
Voz: Jose
Duración: 20.8 s
Idea clave: M6: Kali, auditoría y evidencia.

Narración: M6 es Kali, la estación de auditoría, y no es objetivo. Desde aquí ejecutamos reconocimiento, escaneo, explotación, túneles y captura de evidencia. También aloja el FTP de artefactos y los logs con SHA256. Es el punto de partida y el punto donde se demuestra que todo lo anterior es reproducible.

Apoyo visual: Señalar que M6 no tiene vulnerabilidades objetivo y que su función es auditar y almacenar evidencia.

Transición: Con el mapa claro, dividimos PHP2Shell en seis partes.

---

12 · Reparto de PHP2Shell

Expone: Equipo completo
Voz: Equipo
Duración: 30.9 s
Idea clave: PHP2Shell en seis partes, una por integrante.

Narración: Para la exposición dividimos PHP2Shell en seis partes. Erick presenta la ficha, el CVSS y el version gate. Doris, las precondiciones: NTS, prefork y php.ini. Rony, los offsets reales y el servicio en 8080. Fabiola, la inyección de sesión. Marco, el LFI y la ejecución. Jose cierra con la evidencia, el root por B3, SSH y sudo, y el retest.

Apoyo visual: Recorrer las seis tarjetas y confirmar que cada integrante tiene una parte y una máquina.

Transición: Antes de entrar en W2, veamos la metodología.

---

13 · Metodología y version gate

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 42.1 s
Idea clave: Siete fases; la versión decide si el CVE cuenta.

Narración: Trabajamos en siete fases: alcance, reconocimiento, enumeración, análisis, explotación, remediación y retest. Cada flecha tiene una puerta de versión. Primero verificamos qué versión está instalada; segundo, si está dentro del rango vulnerable; tercero, qué versión la corrige; cuarto, decidimos si es explotable. Si la instalada es igual o mayor que el fix, el CVE no cuenta. En W2 la versión estaba en rango y el servicio explotado cumplía las condiciones: NTS, prefork y offsets reales. El gate confirmó versión y condiciones antes de ejecutar.

Apoyo visual: Señalar las fases y las cuatro casillas del version gate; no entrar aún en W2.

Transición: Este es el inventario que resultó de ese filtro.

---

14 · Qué es PHP2Shell y por qué importa

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 44.4 s
Idea clave: Qué es PHP2Shell y por qué importa.

Narración: PHP2Shell es el caso de dos CVE de corrupción de memoria en PHP 8.4.26 que permiten ejecución remota antes de autenticarse. Se vuelve relevante porque afecta una versión muy extendida, el ataque es remoto y de baja complejidad, y el PoC es reproducible; además, la historia se entiende rápido: de un PHP vulnerable a root. En nuestro caso, la versión 8.4.26 estaba en rango, el servicio 8080 cumplía las condiciones y Chain A dio www-data; luego B3, SSH y sudo dieron root. Por eso el version gate, la evidencia y el retest son parte de la misma cadena.

Apoyo visual: Explicar qué es, por qué llama la atención y cómo se conecta con la cadena; no entrar aún en offsets.

Transición: Con el caso claro, veamos la ficha técnica.

---

15 · W2 · Ficha

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 46.8 s
Idea clave: W2: CVSS 9.8, RCE pre-auth y parche 8.4.27.

Narración: W2 son dos CVE de corrupción de memoria en PHP 8.4.26: CVE-2026-109618 y CVE-2026-109619. El vector es de red, sin autenticación y sin interacción; el CVSS v3.1 es 9.8, crítico. La versión afectada va de 8.4.0 a 8.4.26 y la corrige 8.4.27. El EPSS está sin dato porque el CVE es reciente. El resultado real: RCE como www-data y root por la cadena B3, SSH y sudo.

Apoyo visual: Señalar CVE, CVSS, versión que corrige y estado; aclarar que no se mezclan métricas v3.1 y v4.0.

Transición: La versión parece vulnerable, pero hay que comparar.

---

16 · W2 · Version gate

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 25.1 s
Idea clave: 8.4.26 dentro del rango; 8.4.27 corrige.

Narración: El version gate confirma que 8.4.26 está dentro del rango vulnerable y que 8.4.27 corrige. El servicio expuesto corre con NTS y prefork, y cumple las condiciones del PoC. Por eso declaramos versión y condiciones verificadas, y ejecutamos la cadena. El gate separa hay un CVE de se puede explotar aquí.

Apoyo visual: Señalar la línea 8.4.0–8.4.26–8.4.27 y la decisión: versión sí, condiciones no, hasta ejecutar.

Transición: Veamos esas condiciones.

---

17 · W2 · Precondiciones

Expone: M2 · Doris Apaza Anahua
Voz: Doris
Duración: 29.4 s
Idea clave: Seis precondiciones de build y configuración.

Narración: El servicio explotado presenta seis condiciones. Necesita NTS, es decir, sin Thread Safety; Apache con MPM prefork; display_errors encendido; memory_limit de 16 megabytes; session.upload_progress habilitado y con cleanup apagado; y los offsets del binario real. Se verificaron con php -i, httpd -V y nm antes de ejecutar. El PoC usa comandos de hasta catorce bytes y Chain A suma una sesión compartida y un sink LFI.

Apoyo visual: Recorrer las seis condiciones del servicio: NTS, prefork, display_errors, memory_limit, session.upload_progress y offsets.

Transición: Así se presenta el servicio en 8080.

---

18 · W2 · Rebuild

Expone: M2 · Doris Apaza Anahua
Voz: Doris
Duración: 24.3 s
Idea clave: PHP 8.4.26 NTS en el servicio 8080.

Narración: El servicio vulnerable es PHP 8.4.26 NTS sobre Apache 2.4.49 prefork, publicado en el puerto 8080. Presenta display_errors, memory_limit de 16M, session.upload_progress y session2.php con HTTP 200 y sesión compartida. Es el entorno que se explotó y quedó documentado en EV-W2-01.

Apoyo visual: Señalar un solo servicio en 8080; no hablar de puertos alternos ni de montaje.

Transición: Con el entorno listo, ajustamos el php.ini y los offsets.

---

19 · W2 · Configuración y offsets

Expone: M3 · Rony Churata Huacani
Voz: Rony
Duración: 23.7 s
Idea clave: Offsets derivados del libphp.so real.

Narración: Los offsets no se copian de Internet. Se derivan del libphp.so del servidor M1. Con nm obtuvimos las direcciones de zif_system, zval_ptr_dtor y htdata. Un offset de otro build hace fallar el PoC aunque la versión sea correcta. Por eso el php.ini y los símbolos se verifican sobre el handler de Apache, no solo por línea de comandos.

Apoyo visual: Señalar los tres símbolos y la salida de nm; insistir en que son del binario real.

Transición: Ahora sí, la cadena que convierte la sesión en RCE.

---

20 · W2 · Chain A

Expone: M4 · Fabiola Ancco Rafael
Voz: Fabiola
Duración: 40.5 s
Idea clave: Chain A: sesión, LFI y RCE como www-data.

Narración: Chain A convierte la sesión en ejecución remota en cinco pasos. Primero, session.upload_progress crea /tmp/sess con el contenido del request. Segundo, el nombre del campo y el valor inyectan código PHP. Tercero, el sink LFI incluye ese archivo. Cuarto, PHP interpreta el código en el worker. Quinto, id devuelve uid=33, www-data, y escribimos el marcador PWNED-M1-www-data. La cadena se ejecuta contra el servicio 8080 y luego se escala por B3, SSH y sudo.

Apoyo visual: Seguir los cinco pasos; señalar la salida RCE CONFIRMED y el uid=33(www-data).

Transición: Estos son los comandos exactos.

---

21 · W2 · Comandos

Expone: M5 · Marco Nina Cruz
Voz: Marco
Duración: 28.3 s
Idea clave: Cinco comandos, cinco resultados esperados.

Narración: Estos son los cinco comandos clave. El primero crea la sesión con upload_progress y deja el código inyectado en /tmp/sess. El segundo fuerza el mismo worker y confirma que la sesión persiste. El tercero incluye el archivo de sesión por el LFI. El cuarto ejecuta id y devuelve www-data. El quinto reutiliza la clave de B3 para entrar por SSH como ubuntu y escalar a root con sudo. Cada paso tiene su salida esperada.

Apoyo visual: No leer los comandos completos: explicar qué hace cada uno y qué salida se espera. La demo en vivo los ejecuta.

Transición: W2 no fue la única vía a www-data.

---

22 · Otra vía · W3

Expone: M1 · Erick Ramos Vera
Voz: Erick
Duración: 30.2 s
Idea clave: W3: Apache 2.4.49 y traversal a /bin/sh.

Narración: W3 es una segunda vía, independiente de PHP2Shell. Apache 2.4.49 permite traversal de rutas; con CGI habilitado, la ruta llega a /bin/sh. El comando se ejecuta como www-data. No necesitamos W3 para el resultado, pero demuestra que había más de un camino al mismo nivel. W3 y W2 se parchean por separado.

Apoyo visual: Señalar el comando y la salida uid=33; aclarar que es una ruta alternativa, no la cadena principal.

Transición: La llave para pasar de www-data a root estaba en los respaldos.

---

23 · Otra vía · B3

Expone: M3 · Rony Churata Huacani
Voz: Rony
Duración: 28.6 s
Idea clave: B3: FTP anónimo, Admin13 y root por sudo.

Narración: B3 es la llave maestra. El FTP de M3 permite acceso anónimo y expone /backups. Dentro está credenciales.txt con Admin13. Esa clave se reutiliza en ubuntu en M1. Entramos por SSH, ejecutamos sudo id y obtenemos uid=0 root. No es un exploit nuevo: es reutilización de credenciales. Y es lo que convierte el acceso www-data en control total.

Apoyo visual: Señalar el archivo credenciales.txt, la reutilización en ubuntu y la salida uid=0(root).

Transición: Todo esto tiene que quedar demostrado.

---

24 · W2 · Evidencia

Expone: M6 · Jose Paye Mamani
Voz: Jose
Duración: 28.6 s
Idea clave: EV-W2-07, EV-W2-09 y hashes.

Narración: La evidencia es lo que sostiene todo. EV-W2-07 documenta Chain A: session injection más LFI, con uid=33 www-data desde M1. EV-W2-09 documenta el root: B3, credenciales, SSH como ubuntu, sudo y el archivo /root/PWNED-M1-root.txt. Cada log tiene captura y SHA256.

Apoyo visual: Señalar los dos identificadores y los hashes; evidencia verificable, sin intentos fallidos.

Transición: Este es el impacto real.

---

25 · W2 · Impacto

Expone: M6 · Jose Paye Mamani
Voz: Jose
Duración: 25.1 s
Idea clave: W2 dio www-data; root vino por B3/SSH/sudo.

Narración: El impacto real es este. W2 por Chain A dio www-data. W3 también dio www-data. B3 más SSH y sudo dieron root. La diferencia entre hay un CVE y se explotó está en la evidencia: un id, un archivo o una captura verificable. W2 no dio root por sí solo; la cadena completa sí.

Apoyo visual: Comparar las filas: W2 y W3 a www-data, W1 sin RCE, B3/SSH/sudo a root. No confundir alcance con impacto.

Transición: Veamos la cadena completa en una sola línea.

26 · Cadena completa: de Kali a root

Expone: M1 Erick · M3 Rony · M6 Jose
Voz: Equipo
Duración: 29.1 s
Idea clave: Cinco eslabones de Kali a root.

Narración: La cadena completa tiene cinco eslabones. Kali entra a M1 por W2 Chain A y obtiene www-data. Desde M3, el FTP anónimo entrega credenciales.txt. Esa clave permite SSH como ubuntu en M1. Con sudo llegamos a root y dejamos PWNED-M1-root.txt. La réplica en vivo sigue el mismo orden: versión, precondiciones, PoC, evidencia y retest.

Apoyo visual: Seguir los cinco números; conectar cada eslabón con su evidencia y su máquina.

Transición: El resto de la red también aporta a la cadena.

---

27 · Resto de la red

Expone: M2 Doris · M3 Rony · M4 Fabiola · M5 Marco
Voz: Equipo
Duración: 31.6 s
Idea clave: M2–M5: las vías que sostienen la cadena.

Narración: El resto de la red también aporta. En M2, Redis sin autenticación y las claves de la base permitieron volcar datos. En M3, el FTP anónimo y NFS no_root_squash entregaron credenciales y escritura root. En M4, la clave débil del administrador y el AXFR llevaron a SYSTEM y al mapa de DNS. En M5, VNC sin autenticación abrió el escritorio. M5 sigue pendiente de remediación por filtrado.

Apoyo visual: Recorrer M2, M3, M4 y M5; señalar que M5 está pendiente y no se declara remediado.

Transición: Pasamos a la remediación.

---

28 · Remediación

Expone: Equipo completo
Voz: Equipo
Duración: 38.0 s
Idea clave: PHP 8.4.27, WP 7.0.2, Apache 2.4.51, Redis 6.2.7.

Narración: El parcheo se aplicó en cada servidor. Cambiaron versiones y configuraciones: PHP pasó a 8.4.27, WordPress a 7.0.2, Apache a 2.4.51 y Redis a 6.2.7. Apagamos upload_progress, eliminamos el LFI, cerramos el FTP anónimo y deshabilitamos SMBv1. M1 a M4 quedaron cerrados. M5 no se pudo corregir porque 445, 3389 y 5900 quedaron filtrados; se declara pendiente, no remediado.

Apoyo visual: Explicar qué se cambió y el estado de cada servidor; no detallar el acceso usado para aplicar el parche.

Transición: El retest confirma qué cambió.

---

29 · Retest: antes y después

Expone: Equipo completo
Voz: Equipo
Duración: 38.0 s
Idea clave: M1–M4 cerrados; M5 pendiente.

Narración: El retest usó los mismos comandos de validación. M1 tiene ocho cierres: WordPress 7.0.2, PHP 8.4.27, Apache 2.4.51, el LFI en 404 y las claves rotadas. M2 tiene cuatro: Redis con NOAUTH y puertos filtrados. M3 tiene cinco: FTP anónimo en 530, root_squash y SSH rotado. M4 tiene cinco: SMB1 off, signing on, Spooler off y AXFR cerrado. M5 sigue pendiente.

Apoyo visual: Señalar los cinco bloques y los estados; recordar que M5 no se pudo tocar.

Transición: Cerramos con las lecciones del caso.

---

30 · Cuatro lecciones de PHP2Shell

Expone: Equipo completo
Voz: Equipo
Duración: 32.7 s
Idea clave: Versión, offsets, cadena y parche.

Narración: Cuatro lecciones. Primera: la versión es el punto de partida; PHP 8.4.26 habilitó la cadena. Segunda: los offsets son del binario; copiarlos de otro build hace fallar el PoC. Tercera: W2 más B3 dan root; cada eslabón tiene evidencia. Cuarta: parchear sí cambia el juego; 8.4.27, upload_progress off y LFI eliminado se verificaron en el retest. La fortaleza es la trazabilidad de lo demostrado.

Apoyo visual: Recorrer las cuatro lecciones; cerrar con la idea de trazabilidad, no con una lista de CVE.

Transición: Terminamos con el cierre y las preguntas.

---

31 · Cierre y preguntas

Expone: Equipo completo
Voz: Equipo
Duración: 29.1 s
Idea clave: Preguntas y réplica en vivo.

Narración: Con esto cerramos. PHP2Shell quedó demostrado con Chain A, www-data y root por B3, SSH y sudo. W3 fue una segunda vía real. La remediación y el retest están documentados. Ahora respondemos preguntas y, si el docente lo permite, repetimos la réplica en vivo desde Kali, en el mismo orden: version gate, precondiciones, PoC, evidencia y retest. Gracias.

Apoyo visual: Abrir el turno de preguntas; tener a mano los logs EV-W2-07, EV-W2-09 y EV-007-RETEST.

Transición: Fin de la exposición.

Anexo · Réplica en vivo desde Kali

1. **Version gate.** `php -v`, `httpd -V` y `wp core version` en M1; confirmar 8.4.26 y 2.4.49. Mostrar que 8.4.27 es el fix.
2. **Precondiciones.** `php -i | grep -E 'Thread Safety|memory_limit|display_errors|session.upload_progress'` y `nm libphp.so | grep -E 'zif_system|zval_ptr_dtor'`.
3. **Servicio.** Confirmar `session2.php` en `http://10.10.10.11:8080` y el worker prefork.
4. **Chain A.** Ejecutar los cinco pasos de la diapositiva 21; la salida esperada es `uid=33(www-data)` y el marcador `PWNED-M1-www-data`.
5. **Root.** `curl` al FTP de M3, leer `credenciales.txt`, `ssh ubuntu@10.10.10.11`, `sudo id`; la salida esperada es `uid=0(root)` y `PWNED-M1-root.txt`.
6. **Evidencia.** Mostrar `EV-W2-07`, `EV-W2-09` y `sha256sum` de los logs.
7. **Retest.** Repetir los mismos comandos: PHP 8.4.27, LFI 404, Redis `NOAUTH`, FTP `530`; M5 se declara pendiente.